Documentación técnica explicada
Frontend MediPlan

Versión organizada y legible a partir del análisis entregado por Codex

Proyecto	MediPlan – frontend en React
Tipo de documento	Documento técnico de entendimiento rápido
Objetivo	Explicar la estructura, funcionamiento y puntos críticos del frontend
Alcance	Código fuente del frontend, navegación, API, estado, pruebas y deuda técnica

1. Propósito del documento

Este documento resume de forma clara cómo está organizado el frontend de MediPlan, qué hace cada carpeta importante y cuáles son los archivos que conviene revisar primero. La idea es que cualquier desarrollador nuevo pueda entender rápidamente la aplicación sin tener que descubrir todo directamente desde el código.

El contenido fue reorganizado y explicado con un enfoque más práctico. En lugar de limitarse a describir carpetas, también indica cómo se conectan entre sí, qué partes son críticas y dónde están los principales riesgos técnicos.

2. Visión general del proyecto

MediPlan es una aplicación frontend construida con React para una gestión médica básica. Según el código analizado, su objetivo principal es permitir la autenticación del usuario y el acceso a módulos como doctores, pacientes y citas.

La comunicación con el backend se hace por HTTP mediante Axios. La configuración base de esa conexión está centralizada en el archivo frontend/src/api/axiosConfig.js, el cual apunta a la ruta base http://localhost:4000/api.

Desde el frontend se observan llamadas a endpoints para autenticación, consulta de doctores y operaciones CRUD sobre pacientes y citas.

Flujo funcional principal

1. El usuario entra a la ruta / y visualiza la pantalla de login.
2. Cuando inicia sesión correctamente, el sistema guarda token y user en localStorage por medio de AuthContext.
3. Luego el usuario es redirigido a /dashboard.
4. Desde la barra superior o desde el dashboard puede navegar a doctores, pacientes y citas.
5. Las páginas protegidas consumen la API y muestran tablas y formularios con Bootstrap.
6. Cuando el usuario cierra sesión, se limpia localStorage y se bloquea el acceso a rutas privadas.

Endpoints identificados desde el frontend

Método y ruta	Uso observado
POST /auth/login	Inicio de sesión
GET /doctores	Consulta de doctores
GET /pacientes	Listado de pacientes
POST /pacientes	Creación de pacientes
PUT /pacientes/:id	Edición de pacientes
DELETE /pacientes/:id	Eliminación de pacientes
GET /citas	Listado de citas
POST /citas	Creación de citas
PUT /citas/:id	Edición de citas
DELETE /citas/:id	Eliminación de citas

3. Estructura general del proyecto

Dentro de la carpeta frontend se observan varias carpetas y archivos, pero no todos tienen el mismo peso. Algunos son generados automáticamente, mientras que otros son críticos para entender el funcionamiento real de la aplicación.

Elemento	Función	Importancia
build/	Salida generada para producción	Útil para despliegue, no para leer lógica
coverage/	Reportes de pruebas	No es crítica para ejecutar la app
node_modules/	Dependencias instaladas	Necesaria para desarrollo, pero generada
public/	Plantilla HTML y recursos públicos de CRA	Crítica para el arranque
src/	Código fuente principal	Es la carpeta más importante
package.json	Scripts y dependencias	Crítico
README.md	Documentación del repositorio	Actualmente genérica

Elementos más críticos

- src/index.js: punto de entrada de React.
- src/App.js: enrutamiento y organización de la aplicación.
- src/api/axiosConfig.js: configuración compartida de la API.
- src/context/AuthContext.jsx: manejo global de autenticación.
- public/index.html: contenedor base donde React monta la aplicación.
- package.json: dependencias, scripts y herramientas del proyecto.

4. Explicación de carpetas y archivos clave

src/

Contiene la lógica principal del frontend.

Desde aquí se montan las rutas, el contexto de autenticación, los componentes compartidos, las páginas y las pruebas.

Los archivos más importantes son index.js, App.js, setupTests.js, index.css y las subcarpetas api, components, context, pages, tests y utils.

src/api/

Guarda la configuración HTTP compartida.

El archivo axiosConfig.js crea una instancia de Axios, define la URL base y agrega el token de autenticación al header Authorization si existe en localStorage.

No hay una carpeta services separada; por eso las páginas llaman a la API directamente.

src/components/

Contiene componentes compartidos de estructura y navegación.

Layout.jsx envuelve las páginas privadas y NavbarMediPlan.jsx ofrece navegación superior y cierre de sesión.

La lógica de negocio compleja no vive aquí; está concentrada sobre todo en pages/.

src/context/

Contiene el estado global de autenticación.

AuthContext.jsx expone user, login y logout, y además sincroniza la sesión con localStorage.

Es una pieza central porque PrivateRoute depende de este contexto para permitir o bloquear el acceso a rutas privadas.

src/pages/

Aquí están las vistas principales del sistema.

Login.jsx maneja la autenticación; Dashboard.jsx muestra la vista principal; Doctores.jsx consulta doctores; Pacientes.jsx y Citas.jsx implementan operaciones CRUD.

Es la carpeta más importante desde el punto de vista funcional, porque concentra UI, estado local y llamadas HTTP.

src/tests/

Contiene pruebas organizadas por tipo.

Se observaron subcarpetas unit e integration, además de un App.test.js fuera de esa estructura.

Esto indica que sí existe una base de pruebas, aunque la organización todavía no es totalmente uniforme.

src/utils/

Contiene utilidades auxiliares.

Se identificó alerts.js como una capa para encapsular SweetAlert2 con helpers como alertSuccess, alertError y alertConfirm.

Aunque está disponible, no se ve integrada de forma consistente en las páginas actuales.

public/

Contiene archivos estáticos heredados de Create React App.

index.html define el contenedor root. También se observan manifest.json, favicon.ico, robots.txt y logos por defecto de CRA.

Aún conserva configuraciones genéricas como el título React App.

Carpetas que no se observan en este frontend

Durante el análisis no se encontraron carpetas como services/, hooks/, routes/ o assets/ dentro de src. Esto es importante porque muestra que la arquitectura es simple y directa, pero también más acoplada:

- No existe una capa de servicios desacoplada para centralizar todas las llamadas HTTP.
- No se observan hooks personalizados para encapsular lógica reutilizable.
- Las rutas están definidas directamente en App.js.

- No hay una carpeta dedicada para recursos gráficos dentro de src.

5. Páginas y navegación

Las páginas están definidas en src/pages y el enrutamiento se concentra completamente en src/App.js. El proyecto utiliza BrowserRouter, Routes, Route y un componente local llamado PrivateRoute.

PrivateRoute consulta AuthContext. Si no hay usuario autenticado, redirige a /. Si sí lo hay, permite la carga de las páginas privadas dentro de Layout.

Rutas observadas

- / → Login
- /dashboard → Dashboard
- /doctores → Doctores
- /pacientes → Pacientes
- /citas → Citas

Lectura práctica de la navegación

- Login es la puerta de entrada al sistema.
- Dashboard funciona como panel principal y punto de distribución hacia los demás módulos.
- NavbarMediPlan ofrece navegación superior entre módulos protegidos.
- Pacientes, Doctores y Citas incluyen además un botón para volver al dashboard.

Limitaciones visibles en la navegación

- No se observó una ruta comodín tipo * para manejar errores 404.
- Las rutas no están desacopladas en un módulo propio.
- El módulo de doctores parece implementar solo consulta, no un CRUD completo en frontend.

6. Componentes y reutilización

La carpeta src/components contiene principalmente componentes de estructura general. En este proyecto no se observa una biblioteca de componentes reutilizables más granular ni una separación clara entre componentes visuales pequeños y componentes contenedores.

- Layout.jsx: wrapper para páginas privadas.
- NavbarMediPlan.jsx: navegación superior y cierre de sesión.
- La lógica más fuerte permanece en pages/, no en components/.

7. Consumo de API

El acceso al backend está centralizado solo a nivel de configuración, no a nivel de servicios. Esto significa que existe un único punto de configuración de Axios, pero las llamadas concretas se hacen directamente desde las páginas.

- Login.jsx hace la petición de autenticación.
- Doctores.jsx consulta el listado de doctores.
- Pacientes.jsx implementa GET, POST, PUT y DELETE sobre pacientes.

- Citas.jsx implementa GET, POST, PUT y DELETE sobre citas y además consulta doctores y pacientes.

Aspectos importantes del diseño actual

- La URL base está fija en http://localhost:4000/api.
- El token se toma desde localStorage.
- El header Authorization se agrega mediante un interceptor de request.
- No se observaron variables de entorno, interceptores de respuesta, refresh token ni manejo centralizado de errores.

8. Manejo de estado

El proyecto combina dos niveles de estado. Por un lado, usa Context API para la autenticación global. Por otro, cada página administra su propio estado local mediante useState y useEffect.

- AuthContext.jsx controla user, login y logout.
- La sesión se persiste en localStorage.
- Pacientes, Doctores y Citas mantienen su estado dentro de cada vista.
- No se observaron Redux, Zustand, React Query, SWR ni hooks personalizados para manejo de datos.

Un punto sensible es que el frontend parece considerar válida la sesión mientras exista información de usuario en localStorage. No se observa validación activa del token al arrancar la aplicación.

9. Pruebas

Sí existen pruebas en el frontend. Se observan pruebas unitarias y de integración, además de la configuración base para el entorno de pruebas.

- src/App.test.js: prueba básica de render.
- src/tests/unit/AuthContext.test.jsx: prueba unitaria del contexto de autenticación.
- src/tests/integration/App.login.int.test.jsx: prueba de integración del flujo de login.

Herramientas y observaciones

- Se observan Jest y React Testing Library.
- setupTests.js incluye mocks globales para Axios y SweetAlert2.
- Existe carpeta coverage/ con reportes generados.
- La cobertura todavía es limitada en módulos como pacientes, citas, doctores, rutas protegidas y escenarios de error.

10. Dependencias relevantes

De acuerdo con package.json, estas son las librerías más importantes del proyecto y su papel dentro de la aplicación:

Dependencia	Uso principal
react y react-dom	Base del frontend.
react-router-dom	Navegación SPA y protección de rutas.

axios	Cliente HTTP para comunicación con el backend.
bootstrap	Estilos y layout base.
bootstrap-icons	Iconografía.
sweetalert2	Alertas modales.
react-scripts	Toolchain de Create React App.
@testing-library/* y jest-dom	Pruebas de interfaz y componentes.
web-vitals	Métricas heredadas de CRA.

También se observó que react-bootstrap está instalada, pero no parece usarse en el código actual. De forma similar, alerts.js existe, pero no se integra de manera visible en las páginas analizadas.

11. Flujo técnico resumido

7. src/index.js importa estilos globales, Bootstrap e íconos, y monta la aplicación React.
8. App.js envuelve el sistema en AuthProvider y define el router.
9. PrivateRoute consulta el contexto de autenticación para proteger páginas privadas.
10. Layout agrega la barra superior y estructura común de vistas protegidas.
11. Cada página administra su estado local y hace sus propias llamadas HTTP.
12. AuthContext mantiene sincronizada la sesión entre React y localStorage.

12. Riesgos, deuda técnica y puntos sensibles

Punto sensible	Explicación
URL del backend hardcodeada	La baseURL está fija a localhost, lo que complica despliegues en otros ambientes.
Acoplamiento entre UI y acceso a datos	Pacientes y Citas mezclan render, estado, formularios y llamadas HTTP en un mismo archivo.
Duplicación de patrones	Se repiten estructuras de loading, mensajes, handleSubmit, handleDelete y recargas de tablas.
Cobertura funcional parcial en doctores	El dashboard habla de gestión, pero en frontend se observa solo listado.
Sesión validada solo por localStorage	No se ve validación activa del token, manejo de expiración ni refresh token.
Pruebas con estructura poco uniforme	Hay pruebas dentro y fuera de src/tests.
Restos de Create React App	Persisten README, título, manifest y assets genéricos de CRA.
Import redundante de Bootstrap	Se observó import duplicado de bootstrap/dist/css/bootstrap.min.css.
Dependencias o utilidades subutilizadas	react-bootstrap y alerts.js están presentes, pero no integrados de forma visible.
Cobertura de pruebas limitada	No se ve cobertura profunda de CRUD, errores de red

	ni navegación protegida completa.

13. Guía rápida para entender el proyecto

Si alguien recibe este frontend por primera vez, este sería el orden recomendado para revisarlo:

13. Leer el README del repositorio raíz para tener el contexto general.
14. Abrir src/index.js para entender cómo arranca la aplicación.
15. Revisar src/App.js para comprender rutas y protección.
16. Leer src/context/AuthContext.jsx para entender la sesión.
17. Revisar src/api/axiosConfig.js para identificar cómo se conecta con el backend.
18. Entrar a Login.jsx como puerta de entrada funcional.
19. Luego revisar Pacientes.jsx y Citas.jsx, porque ahí está gran parte de la lógica real del frontend.

Archivos que conviene revisar primero

- frontend/src/index.js
- frontend/src/App.js
- frontend/src/context/AuthContext.jsx
- frontend/src/api/axiosConfig.js
- frontend/src/components/Layout.jsx
- frontend/src/components/NavbarMediPlan.jsx
- frontend/src/pages/Login.jsx
- frontend/src/pages/Dashboard.jsx
- frontend/src/pages/Pacientes.jsx
- frontend/src/pages/Citas.jsx
- frontend/src/pages/Doctores.jsx

Carpetas que conviene entender antes de modificar código

- src/pages/: concentra la lógica funcional.
- src/context/: controla autenticación y rutas privadas.
- src/api/: define la comunicación con el backend.
- src/components/: impacta la estructura común de las vistas.
- src/tests/: ayuda a ampliar cobertura sin romper la base existente.

14. Conclusión ejecutiva

El frontend de MediPlan es una SPA pequeña, directa y relativamente fácil de recorrer. Su fortaleza principal es la simplicidad: pocas capas, rutas claras y una estructura que un desarrollador puede seguir sin demasiadas abstracciones.

Sin embargo, esa misma simplicidad también genera acoplamiento. La lógica de negocio, el estado local y las llamadas a la API están muy concentradas en las páginas. Por eso, aunque el proyecto sea accesible para una primera lectura, también tiene oportunidades claras de mejora en organización, pruebas y preparación para despliegue.

Para entenderlo rápido, los archivos más importantes son src/index.js, src/App.js, src/context/AuthContext.jsx, src/api/axiosConfig.js y la carpeta src/pages/.